Sparse hard oblique forests for jet classification

Architecture, TAO training, and Select–Freeze–Refit

Tree benchmark implementation note

26 October 2026

Abstract

We document the floating-point hard oblique-tree implementation used in this benchmark, its Tree Alternating Optimization (TAO) training procedure, and a two-stage use of sparsity: select coefficients with an L1 penalty, freeze their support, then refit the surviving coefficients without shrinkage. We call this procedure Select–Freeze–Refit (SFR), a descriptive project name rather than a claim of a new general selection principle. On the fixed quark/gluon versus W/Z/top validation task, the implementation default is a three-tree depth-four forest with linear leaves, 418 nonzero coefficients in 1,581 dense slots, and 56.20% signal efficiency at 1% background acceptance. The XGBoost reference gives 55.48% with 3,099 scalar slots. A higher-efficiency 837-coefficient alternative reaches 57.12%; a one-tree reference reaches 55.63%. These are validation-selected floating-point results, not held-out test or synthesized FPGA results. Actual cut equations, complete tree printouts, training settings and code entry points are included for implementation and reproduction.

Implementation target. Dorian's pinned default is `fpga_handoff/v3/compact_refit_3t_d4/`, selected explicitly for compression, not for the highest efficiency. The authoritative pointer is `fpga_handoff/CURRENT_MODEL.json`. The coefficient snapshot is full-refit pass 8; the source full-refit stage subsequently converged at pass 15. The selected pass itself is *not* labelled stationary. No coefficients or training policies were changed to produce this note.

Contents

1 Scope, dataset and evidence **3**

2 Oblique-tree architecture as implemented **3**
2.1 Hard routing and complete binary storage . . . 3
2.2 Linear leaves, reference logits and forest aggregation . . . 4
2.3 Parameter accounting and preprocessing . . . 4

3 TAO training and numerical parameters **5**
3.1 One normalized objective for all blocks . . . 5
3.2 Leaf subproblems and the joint forest refit . . . 5
3.3 The reduced decision-node problem . . . 6
3.4 An epoch, initialization and forest coupling . . . 6

4 An actual printed tree and its affine cuts **7**

5 Training considerations and Select–Freeze–Refit **8**
5.1 Separate support selection from shrinkage . . . 8
5.2 The exact Compact refit training chain . . . 9
5.3 Solver budgets and the numerical meaning of convergence . . . 9
5.4 Separation and held fits . . . 10
5.5 Penalty strength, cut-scale ambiguity and precision . . . 11

6 Comparison with XGBoost and current results **11**
6.1 Reference configuration and metric . . . 11
6.2 Pinned v3 models . . . 12
6.3 The fixed-support penalty study . . . 13
6.4 Uncertainty, running studies and timing limits . . . 14

7 Reproduction, code map and the firmware handover **15**
7.1 Pinned environment and entry points . . . 15
7.2 Frozen coefficients and reference verification . . . 15

8 Conclusions and implementation priorities **16**

References **16**

A Input feature order and model identity **18**

B Complete Compact refit forest printout **18**
B.1 Tree 0 . . . 19
B.2 Tree 1 . . . 21
B.3 Tree 2 . . . 23

1 Scope, dataset and evidence

The intended application is a compact classification function for a future FPGA implementation in the CMS Phase-II Level-1 trigger context. The present study addresses the model and its training in floating point. It does not yet establish a fixed-point format, device, resource count, clock, initiation interval or hardware latency. Neural networks are not trained or compared in this note.

The implementation and frozen handover are maintained in the project repository [1]. The primary example uses 16 high-level features of jets from the dataset catalogued as OpenML 42468, `hls4ml_lhc_jets_hlf` [2]. The benchmark uses archived arrays supplied with the project; it does not fetch a new dataset or resplit OpenML rows at training time. Class 0 is quark/gluon background; class 1 is W/Z/top signal. The unchanged partitions contain 531,200 training events, 132,800 validation events and 166,000 held-out test events. There are 52,511 background and 80,289 signal events in validation. The split and model seed are 42.

`configs/bdt_qg_vs_wzt_seed42.yaml` identifies `data/qg_vs_wzt_standardized.npz` and `data/qg_vs_wzt_metadata.json`. The standardization parameters are fitted on training only. All algorithms use these same exported train/validation arrays, labels and feature order. The oblique-tree scans, SFR selection and comparisons in this note use validation only. An earlier XGBoost reproduction check did evaluate the 166,000-event test partition; that historical check is documented in the root README. Thus this partition is not wholly uninspected. Its scores have not been used to select the oblique architectures or penalties reported here. Other exported tasks are quark/gluon versus top and multiclass classification; the results and binary firmware interface in this note concern the primary quark/gluon versus W/Z/top task only.

The data file is pinned by SHA-256:

`f5ec6f7ed442010656a904700f8f6421cc34b674a1ec810c5cf6b2de003ed926`

Code checks in `scripts/audit_tao_splits.py` and the training workers guard this split contract. The manuscript input generator reads validation arrays only; hashing the complete archive does not itself evaluate its test arrays.

What is frozen and what is still research? The v3 handover, comparison timings and SFR results were frozen on 25 October 2026. A separate research check on 26 October re-evaluated saved architecture-scan checkpoints and recorded their hashes in `docs/handover_status_2026-10-26.json`. That check found 37 of 58 architecture-scan cases converged; the 16-point L1 path study was complete. The ten-case SFR study had six converged full refits and four held outcomes. Later running-job scores are not silently substituted into the frozen v3 comparison. Code and asset fingerprints used for this note are recorded in `sources/provenance.json` inside this standalone document folder.

2 Oblique-tree architecture as implemented

2.1 Hard routing and complete binary storage

`src/tao/hard.py` defines `HardTree` and `HardTAOClassifier`. For feature vector $x \in \mathbb{R}^F$, an internal node j of tree t evaluates an affine cut

$$h_{tj}(x) = w_{tj}^{\mathsf{T}} x + b_{tj}, \qquad \text{right if } h_{tj}(x) > 0, \quad \text{left if } h_{tj}(x) \le 0. \tag{1}$$

An axis-aligned cut is the special case with one nonzero feature weight. A genuinely oblique cut can combine several correlated features. The word “hard” refers to a discrete branch choice, not a sigmoid mixture of the two subtrees. There is no soft routing in the models reported here.

A tree of depth D stores $I = 2^D - 1$ internal nodes and $L = 2^D$ leaves. Depth counts decisions on a full root-to-leaf path. The root has heap index 0; children of internal node j are $2j + 1$ (left) and $2j + 2$ (right). A leaf has array index $\ell =$ heap index $- I$. Some complete-array branches may be logically unreachable because a split is input-independent. A zero weight vector with $b \leq 0$ always chooses left; with $b > 0$ it always chooses right. This does not depend on the empirical occupancy of a branch in a finite sample.

The stored arrays are `split_w` with shape (I, F), `split_b` with shape $(I,)$, and `leaf_parameters` with shape $(L, B, K - 1)$ for K classes. Here $B = F + 1$ for linear leaves and $B = 1$ for constant leaves. `HardTree.routes`, `HardTree.leaf_indices` and `HardTree.predict_subtree` implement the traversal. The selected models use the `dense` NumPy reference traversal, not a generated sparse firmware kernel.

2.2 Linear leaves, reference logits and forest aggregation

The final class is the fixed reference class, with logit zero. A linear leaf stores $K - 1$ affine functions,

$$f_{t\ell k}(x) = a_{t\ell k}^{\mathsf{T}} x + c_{t\ell k}, \qquad k = 0, \ldots, K - 2. \tag{2}$$

For constant leaves, $f_{t\ell k}(x) = c_{t\ell k}$. If $\ell_t(x)$ is the selected leaf, the forest forms

$$z_k(x) = \sum_{t=0}^{T-1} f_{t,\ell_t(x),k}(x), \quad z_{K-1}(x) = 0, \qquad p_k(x) = \frac{e^{z_k(x)}}{\sum_{q=0}^{K-1} e^{z_q(x)}}. \tag{3}$$

This is the convention of `HardTAOClassifier.decision_function` and `HardTAOClassifier.predict_proba`. The logits are summed, not averaged; probabilities are not summed. Fixing a reference class removes a redundant class-logit degree of freedom, although different forest parameterizations can still yield identical summed logits. The input and logit calculations use binary64; the public `predict_proba` array is cast to float32. The reported ROC evaluation uses the actual API scores, including any resulting probability ties, while the portable margin interface remains binary64.

For the binary task, the single stored free logit is the *background* logit. Thus

$$m(x) = -z_0(x) = -\sum_t f_{t,\ell_t(x),0}(x), \qquad p_{\text{signal}}(x) = \text{sigmoid}(m(x)). \tag{4}$$

Firmware can compare the increasing signal margin m directly with a threshold. Reversing the sign would reverse the classification. The training code supports multiclass leaf fits, but the current `tree-benchmark.binary-hard-oblique.v1` interchange schema and C++ reference are binary; they must be extended before a multiclass firmware export.

Linear leaves model first-order dependence *within* each region. They do not impose continuity across a split: two affine leaf functions can jump at a boundary. A forest is affine on each intersection of its trees' leaf regions; adding trees refines that partition and can improve a small forest beyond what extra depth in one tree achieves. Conversely, the inclusion of axis-aligned cuts as a special case does not prove that three shallow oblique trees can represent every function of a hundred-tree BDT. The comparison is empirical, not a containment theorem between those two fixed architectures.

2.3 Parameter accounting and preprocessing

Counting scalar coefficients and biases, but excluding feature indices, topology, masks, preprocessing and metadata, gives

$$P_{\text{dense}} = T\left[(2^D - 1)(F + 1) + 2^D B(K - 1)\right]. \tag{5}$$

For $F = 16$, $K = 2$ and $D = 4$, a linear-leaf tree has $15 \cdot 17 + 16 \cdot 17 = 527$ slots; three have 1,581. A constant-leaf tree has $15 \cdot 17 + 16 = 271$ slots. These allocated slots must not be confused with the number of coefficients that are exactly nonzero.

The Compact refit stores 418 nonzero coefficients, of which 379 remain after removing coefficients in algebraically unreachable branches. Dense serialized arrays still contain all 1,581 slots. `src/tao/sparsity.py`, function `sparsity_profile`, counts exact zeros and safely follows constant routes; it does not use validation occupancy to prune rare branches.

The primary training arrays are already standardized, so the classifier's `InputTransform` is identity. The utility `InputTransform.from_variances` correctly uses square roots of variances for a non-identity scaler. Applying a second standardization would change the model. For a raw-input export, `export_model` in `src/tao/portable.py` folds the saved scaler into every split and linear leaf:

$$x_f = \frac{r_f - \mu_f}{s_f}, \qquad w_f^{\text{raw}} = \frac{w_f}{s_f}, \qquad b^{\text{raw}} = b - \sum_f \frac{w_f \mu_f}{s_f}. \tag{6}$$

"Raw" denotes unstandardized high-level jet features, not detector inputs. Folding preserves feature-weight zeros for nonzero scales, but can change bias values and bias zero counts. The exact feature order is in Appendix A.

3 TAO training and numerical parameters

TAO revisits the parameters of an existing tree instead of making only a single greedy top-down pass. The original work describes node-wise alternating optimization and sparse oblique splits [3]; subsequent work uses sparse oblique trees to inspect learned feature representations [4]. Our code is an adaptation for an additive forest with constant or affine multiclass logits and cross-entropy. The precise objective, acceptance tests and finite numerical stopping rules below describe our implementation; they should not be replaced by stronger claims about global hard-tree optimization.

3.1 One normalized objective for all blocks

For events (x_i, y_i) with nonnegative weights v_i and $W = \sum_i v_i$, the implementation minimizes

$$\begin{aligned} J(\theta) = \frac{1}{W} \sum_i v_i \, \mathrm{CE}(y_i, z(x_i)) + \lambda_s \sum_{t,j} \left(\|w_{tj}\|_1 + |b_{tj}| \right) \\ + \lambda_\ell \sum_{t,\ell,k} \|a_{t\ell k}\|_1 , \qquad \mathrm{CE}(y, z) = \log \sum_k e^{z_k} - z_y. \end{aligned} \tag{7}$$

In this benchmark $v_i = 1$. Split biases are penalized; leaf biases are not. A constant leaf has no feature coefficients, so its nominal leaf L1 strength has no effect under this convention. All node and leaf subproblems retain the *full training-weight sum* W as denominator, not the number of events reaching that node. Otherwise the same printed λ would mean a different regularization strength at different depths or occupancies.

`cross_entropy_rows`, `ce_value_gradient`, `HardTAOClassifier._penalty` and `HardTAOClassifier.objective` implement these definitions in `src/tao/hard.py`. There is no L2 substitution for the requested L1 penalty. Classical Lasso uses squared-error linear regression; here "Lasso-like" refers to L1-penalized classification and selection, not a call to `sklearn.linear_model.Lasso`.

3.2 Leaf subproblems and the joint forest refit

With routes fixed, the leaf logits are linear in their coefficients. For a single tree update, predictions from all other trees are held as a fixed logit offset. Omitting that offset would fit each tree as though it were alone, instead of optimizing the forest loss. With all routes fixed, the complete forest leaf problem is convex, although its parameters need not be uniquely identified.

`fit_leaf_parameters` uses SciPy's `minimize` with L-BFGS-B and an analytic softmax cross-entropy gradient. A penalized scalar is represented as $\theta_j = u_j - v_j$, $u_j, v_j \geq 0$, with penalty $\lambda(u_j + v_j)$.

Unpenalized coordinates are unconstrained real variables. At a finite solver budget, both parts can remain positive. The code therefore canonicalizes their difference and evaluates the *actual stored* $\lambda|u_j - v_j|$, rather than reporting the larger packed penalty as though it were the objective of the exported coefficients. A candidate is retained only if its actual objective does not increase.

`HardTAOClassifier._joint_design` builds a CSR design matrix whose row activates the selected leaf block of every tree. A linear-leaf row stores $T(F+1)$ entries, so the three-tree training problem has 27,091,200 stored design entries for 531,200 events. The implementation constructs CSR arrays directly. It still builds full feature blocks even for masked-out coefficients; model sparsity is not yet fully exploited in this training design. The method `HardTAOClassifier.global_refit` jointly reoptimizes all leaf coefficients, not just the last tree, using that design.

3.3 The reduced decision-node problem

Consider events currently reaching node j; ancestors, descendants and other trees are fixed. Evaluate the full-forest loss obtained by routing an event through the left or right subtree:

$$C_i^L = \mathrm{CE}(y_i, o_i + g_L(x_i)), \quad C_i^R = \mathrm{CE}(y_i, o_i + g_R(x_i)), \quad \Delta_i = C_i^L - C_i^R, \tag{8}$$

where o_i is the logit offset of the other trees. Apart from an additive constant, changing the split minimizes a weighted binary misrouting loss,

$$\frac{1}{W} \sum_{i \in \mathcal{R}_j} v_i |\Delta_i| \mathbb{1}[\mathbb{1}[h_j(x_i) > 0] \neq \mathbb{1}[\Delta_i > 0]] + \lambda_s \left(\|w_j\|_1 + |b_j| \right). \tag{9}$$

The reduced label is *right versus left*, not the original physics class. This binary routing problem is compatible with a multiclass forest loss. Events with $v_i|\Delta_i| = 0$ contain no information about this split and are excluded from the logistic fit; they do not create a third branch.

The code fits a weighted logistic *proposal* for these labels. In ordinary penalized training, `fit_l1_split` can use scikit-learn's `LogisticRegression` with `solver='liblinear'`, an L1 penalty and $C = 1/(\lambda_s W)$, or the same positive/negative L-BFGS-B parameterization. The synthetic LIBLINEAR intercept is penalized consistently with Eq. (7). An unpenalized split uses the L-BFGS-B route, since LIBLINEAR does not supply the requested unpenalized objective. The fixed-support variant uses masked L-BFGS-B directly.

The logistic objective is not the hard routing objective. The selected `split_search='hard_threshold'` implementation therefore considers the current direction, the logistic direction, 25/50/75% blends, the raw logistic proposal and a zero-weight constant direction. For each allowed fixed direction it calls `best_threshold` in `src/tao/split_search.py`. The function sorts projected scores, moves tied values together, enumerates representable threshold intervals, and includes the bias penalty. The result is checked again with the *actual hard-routing loss*. Only a strictly improving hard-objective proposal replaces the current split. This is exact threshold optimization along the proposed directions, not exact optimization over all hyperplanes.

In the fixed-support variant, a current split that already sends every positive-weight event to its lower-loss child attains a pointwise lower bound. `FrozenSupportTAO._split_update` returns the certificate `pointwise_lower_bound` without requiring a finite optimum of an auxiliary logistic fit. Otherwise its certificate concerns only the configured proposal family. With just one pseudo-class, the logistic fit is skipped; current and constant directions are still tested subject to frozen bias constraints. This limited proposal family is another reason not to interpret a stopped run as globally optimal over hyperplanes.

3.4 An epoch, initialization and forest coupling

Bottom-up node order makes cached ancestor reaching sets valid while revisiting descendants. Trees are updated sequentially with refreshed offsets. The code does not currently train all forest

Listing 1: Logical structure of an implemented hard-TAO epoch. Names correspond to the Python methods; this is explanatory pseudocode.

```
cache each tree's current logits; total = sum(cache)
for tree in forest:
    offset = total - cache[tree]
    routes = tree.routes(training_inputs)
    fit each leaf with the other-tree offset fixed
    update internal nodes in descending heap order (bottom-up)
    update cache[tree] and total
HardTAOClassifier.global_refit(all_training_inputs, labels)
check loss, leaf KKT residual, routing changes, inner fits
```

members in parallel, nor does it simply boost a residual while keeping earlier members fixed. The serializable version is `begin_epoch`/`step_epoch` in `src/tao/epoch.py`; it can checkpoint between node updates without repeating a partially finished epoch. Cached logits avoid recomputing the whole forest for every proposal.

For fresh models, `initialize_cart` in `src/tao/initialization.py` uses a CART seed on training data only. Forest members use bootstrap draws within that same training partition for initial diversity; later TAO updates use the full original training sample. Leaf class counts receive a 0.5 pseudocount before conversion to reference-class log odds. Initial per-tree logits are divided by T. Early CART leaves are padded by deterministic left-routing zero tests in the complete storage layout. `append_zero_tree` can grow a forest without changing its current predictions, but this option is not used to obtain the fixed three-tree SFR default. That model is a continuation of a frozen v2 forest, as detailed below.

4 An actual printed tree and its affine cuts

We added the read-only `TreePrinter` class in `src/tao/printing.py`, with methods `print_tree`, `format_tree`, `format_forest` and `records`. It reads the portable model specification. It deliberately lives outside the pinned numerical training files, so adding inspection does not invalidate ongoing training implementation hashes or alter the fitted model.

Listing 2: Print a real tree from the handed-over model, without loading a training pickle. The normal Python environment is described in Section 7.

```
import json
from pathlib import Path
from tao.printing import TreePrinter

spec = json.loads(Path(
    "fpga_handoff/v3/compact_refit_3t_d4/standardized/model.json"
).read_text())
printer = TreePrinter(spec, precision=17)
printer.print_tree(0)  # tree indices are zero-based
```

The CLI `scripts/print_tao_tree.py` prints one tree or the whole forest and can write the result to a file. The output identifies each heap node, its left/right path, structural reachability, the full affine equation and both child destinations. It includes unreachable branches rather than silently hiding them. Only exact-zero terms are omitted. At the default precision, all nonzero binary64 coefficients are represented with 17 significant digits; a smaller precision affects display only.

Here are actual standardized-input cuts from the Compact refit. Subscripts (t, j) denote zero-based tree and internal-node indices. These equations are generated from the model JSON, not transcribed or invented examples. Numerical values in this display are rounded to eight significant digits, hence

the $\simeq$ signs.

$$h_{0,0}(x) \simeq 1\,\texttt{c1_b2_mmdt} + 0.4216204 \tag{10}$$

$$h_{0,1}(x) \simeq 0 \tag{11}$$

$$\begin{aligned} h_{0,2}(x) \simeq\ & 0.088106595\,\texttt{c1_b2_mmdt} + 0.0025365055\,\texttt{c2_b2_mmdt} \\ & + 0.17735909\,\texttt{m2_b1_mmdt} + 0.031223362\,\texttt{mass_mmdt} \\ & + 0.10064081\,\texttt{multiplicity} + 0.019058664 \end{aligned} \tag{12}$$

$$h_{1,1}(x) \simeq -2.3776136\times10^{-11}\,\texttt{multiplicity} - 3.1703281\times10^{-11} \tag{13}$$

$$\begin{aligned} h_{2,0}(x) \simeq\ & 0.45196307\,\texttt{c1_b2_mmdt} - 0.0092294932\,\texttt{m2_b1_mmdt} \\ & + 0.31585632\,\texttt{mass_mmdt} + 0.09903625 \end{aligned} \tag{14}$$

The first cut is still axis-aligned: node $(0,0)$ sends an event right when its standardized `c1_b2_mmdt` exceeds approximately -0.4216204. Node $(0,1)$ has exactly zero weights and bias, so equality always sends it left; its right subtree is structurally unreachable. Node $(0,2)$ combines five features and is genuinely oblique. Tree 2 has an oblique root combining three features. Thus the trained model uses a mixture of axis-aligned, oblique and constant decisions.

The coefficients at node $(1,1)$ are tiny but nonzero. Dividing its inequality by a positive common scale reveals an ordinary cut on standardized `multiplicity`, with the inequality reversed when solving for that feature because its coefficient is negative. Replacing these numbers by zero would turn a real split into a constant route. This is why the support and operation counts do not use an arbitrary magnitude cutoff.

Appendix B records *all three trees*, including every cut and every affine leaf, at nine significant display digits. The exact full-precision human-readable record is `generated/compact_forest_full_precision.txt`; the original JSON and hexadecimal C++ headers remain the implementation authority. Tests independently parse printed expressions, recover all 418 nonzero coefficients exactly at 17 digits, reproduce the 379 reachable-coefficient count, and compare reconstructed forest predictions with the independent portable evaluator. They also check zero/equality routing, constant leaves, tiny terms and absence of mutation.

5 Training considerations and Select–Freeze–Refit

5.1 Separate support selection from shrinkage

L1 regularization has two useful but different effects: it can make a coefficient exactly zero, and it shrinks the remaining values. Once a coefficient is to be implemented anyway, retaining shrinkage may unnecessarily compromise the classifier. Post-selection refitting is a familiar idea; relaxed Lasso separates variable selection from subsequent shrinkage in linear regression [5]. We use that motivation, not its regression theory, for a nonconvex hard forest.

We name our concrete implementation Select–Freeze–Refit:

1. **Select.** Train an L1-penalized parent to the configured numerical stopping criterion. Trace separate split, leaf and joint penalty paths from warm-started *terminal* parents, following the regularization-path principle also used in sparse TAO [3]. Select the penalty/support strategy using validation, never test data.
2. **Freeze.** Copy the parent and define $M_j = \mathbb{1}[\theta_j \neq 0]$ once, including split and leaf biases and all free classes. Preserve the topology and the exact zeros. Remove the split and leaf L1 penalties.
3. **Refit.** First optimize surviving leaf coefficients with every split fixed. Optionally continue by optimizing surviving split and leaf coefficients, still constrained by the original masks. Select among eligible saved checkpoints at the declared validation working point.

Each stage starts with a joint fixed-route leaf refit and repeats that refit after each node-update pass. A pass is not a single optimizer iteration. The full refit solves, approximately and by alternation,

$$\min_{\theta} \frac{1}{W} \sum_i v_i \, \mathrm{CE}(y_i, z(x_i; \theta)) \quad \text{subject to} \quad \theta_j = 0 \text{ whenever } M_j = 0. \tag{15}$$

The surviving coordinates are real unconstrained variables and can change sign. They are not forced to remain nonzero. If a free coefficient becomes zero, its original mask entry still allows it to move later; the mask is not reselected. Simply reducing L1 on the original unrestricted model would instead permit removed coordinates to regrow, defeating the controlled fixed-support study.

`freeze_support` and `continue_full_refit` in `src/tao/support.py` implement this distinction. `FrozenSupportTAO` checks mask hashes, shapes, topology, finite values, zero regrowth and the zero L1 settings. Leaf-only splits are bit-identical to the parent. Splits that were already input-independent retain their decision and bias. A frozen-zero split bias is not revived by an unrestricted threshold search. No epsilon pruning, L2 fallback or implicit renormalization is introduced.

5.2 The exact Compact refit training chain

The default is *not* a fresh fit at its final L1 values. Its chain is:

Stage	Configuration / setting
Frozen start	v2 `winner_3t_d4` portable coefficients
Same-penalty control	`configs/l1_path_v1/control.json`
Split L1 path	$3\times10^{-5}, 10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}$
Leaf L1 during path	10^{-5} throughout
Refit parent	Converged terminal `split_05` checkpoint, 418 nonzeros
SFR leaf-only then full	`configs/support_refit_scan_v1/split300.json`

The path manifest is `configs/tao_l1_path_v1.json`; the individual split configs are `split_01.json` through `split_05.json` under `configs/l1_path_v1/`. The parent worker is `scripts/train_tao_fully.py`. The path configurations start with 300 local leaf iterations, 250 iterations per joint-leaf chunk, and 300 auxiliary split iterations. The worker logs repeated refit chunks and any adaptive inner-budget increases instead of treating an iteration limit as convergence. The refit driver is `scripts/train_tao_support_refit.py`, with evaluated-snapshot archiving installed by `scripts/run_support_refit_scan_case.py` and `scripts/support_refit_artifacts.py`.

For this fixed support, validation efficiency changes from 54.59% for the L1-selected terminal parent to 54.81% after leaf-only refitting. The full stage converges at pass 15 with 55.98%. Its best saved validation checkpoint is pass 8, at 56.20%, with the same 418 nonzero coefficients. A converged source run therefore does not imply that its selected earlier snapshot is stationary. The immutable snapshot is pinned by SHA-256:

`e89b8923fba66f5df02046e04a1828a8ea4c56f60c1d7b201a2a2f92a0ab00fa`

5.3 Solver budgets and the numerical meaning of convergence

The Compact refit uses the settings in Table 1. A solver iteration cap is a compute budget, not a success criterion. The worker repeats joint-leaf solver chunks until the constrained leaf KKT residual meets its threshold, or labels the case unsuccessful if a guard is reached.

`leaf_kkt_residual` and `convergence_check` in `src/tao/convergence.py` implement the checks. For a free unpenalized leaf coordinate, stationarity requires its normalized CE gradient to be small. For a penalized coordinate, the residual checks the L1 subgradient condition; values within 10^{-10} of zero are treated as zero *for this numerical stationarity calculation.* That tolerance is distinct from

Table 1: Default Compact refit settings. Penalties are zero during refitting; the parent was selected using split/leaf L1 $0.003/10^{-5}$.

Setting	Value
Training / split seed	42 / 42
Local leaf iterations per call	400
Joint-leaf iterations per chunk	400 (at most 100 chunks per phase)
Auxiliary split iterations	800
L-BFGS-B `ftol` / `gtol`	10^{-12} / 10^{-8}
Leaf KKT infinity-norm tolerance	10^{-6}
Relative training-objective gain tolerance	10^{-5}
Routing-change fraction tolerance	10^{-4}
Stable passes required / minimum passes	3 / 5 per stage
Maximum passes per stage	200 (guard, not convergence)
Separation diagnostic time limit	600 s
Checkpoint allocation budget	2,700 s; clean return code 75

the exact `!=0` mask and parameter-count rule. Frozen coordinates have equality constraints and are excluded from the free-coordinate KKT check. Explicitly, for leaf coordinate q with normalized unpenalized gradient g_q and applicable penalty λ_q (λ_ℓ for a feature weight, zero for a leaf bias), the residual is

$$r_q = \begin{cases} 0, & q \text{ frozen}, \\ |g_q + \lambda_q \operatorname{sign}(\theta_q)|, & q \text{ free and } |\theta_q| > 10^{-10}, \\ \max(|g_q| - \lambda_q, 0), & q \text{ free and } |\theta_q| \le 10^{-10}. \end{cases} \qquad r_{\mathrm{KKT}} = \max_q r_q. \tag{16}$$

For an unpenalized free coordinate this reduces to $|g_q|$ in either branch. The check certifies numerical stationarity of the convex fixed-route leaf problem; no corresponding differentiable KKT claim is made for discrete routing changes.

For each epoch, the code rejects an objective increase above the numerical allowance 10^{-8}. It requires small relative objective gain, small fraction of events whose leaf changes in *any* forest member, and successful inner fits. All must hold for three consecutive passes after the minimum-stage length. These are numerical stopping conditions for the fixed routes/available proposals, not a proof of a global optimum over all trees or hyperplanes.

One mild-control case exposed a practical failure mode: an auxiliary router repeatedly hit 800 iterations even when its hard proposal was rejected and routing had stopped changing. A tested, recorded checkpoint-policy migration raised that case's auxiliary budget to 4,000 without changing its coefficients, masks, objective or stopping tolerances. The next complete pass solved that router in 2,028 iterations; the control eventually converged after 16 full passes. This is documented in `docs/support_refit_scan_budget_amendments.json` and implemented by `scripts/amend_support_solver_budget.py`. It was not a relaxation of the convergence rule or a change to the Compact refit's 800-iteration setting.

5.4 Separation and held fits

Removing L1 can expose complete or quasi separation: the unpenalized logistic loss may approach its infimum only as some coefficients diverge. An iteration cap or tiny loss change must not be mistaken for a finite well-behaved fit. With routes fixed, `separation_check` in `src/tao/support.py` searches for a free-coefficient direction that makes every training true-class versus other-class logit margin nondecreasing and at least one strictly increasing. The reference class remains zero. Box bounds on this homogeneous diagnostic fix its scale; they do not cap the fitted model coefficients.

The sparse LP uses all positive-weight training events and allowed coordinates. A feasible positive improvement indicates separation; a solver failure, ambiguous numerical result or timeout is inconclusive. Its practical score thresholds distinguish an improvement above 10^{-7} from a near-zero score at or below 10^{-10}, with intermediate values held; feasibility is checked numerically. It is a diagnostic, not a proof of uniqueness of the forest parameters. Certificates are cached only while leaf routes remain unchanged.

Of ten SFR cases, two full stages were held for separation and two for inconclusive 600-second LP timeouts. They remain visible in the results. A completed leaf-only stage can still be used: this is how the 57.12% higher-efficiency alternative is certified. Its later 57.24% full-refit snapshot is diagnostic-only and is *not* the handed-over alternative.

5.5 Penalty strength, cut-scale ambiguity and precision

An L1 setting is not an invariant count of model parameters. In exact arithmetic, for any $c > 0$,

$$\mathbb{1}[w^\mathsf{T} x + b > 0] = \mathbb{1}[cw^\mathsf{T} x + cb > 0], \qquad \|cw\|_1 + |cb| = c(\|w\|_1 + |b|). \tag{17}$$

Thus the outer hard-routing loss is unchanged by a positive rescaling, while its raw-coefficient L1 penalty can be reduced without removing a feature. The logistic proposal gives a margin-scale-dependent subproblem, but the outer hard objective is still not a globally scale-identified model-size criterion. The penalty is therefore an empirical knob in our alternating procedure, not an exact L0 penalty or a theorem that a particular λ selects a unique topology. This observation follows directly from our routing and objective definitions; it is not an additional normalization implemented in the archived models.

The tiny real split in Section 4 illustrates the issue. All reported masks use exact zeros. Renormalizing a hyperplane by a positive scale could be useful before quantization, but must be an explicit separately verified transformation: floating reduction order and finite-precision rounding can matter near boundaries. No such renormalization or magnitude threshold was silently applied to v3. The printout even contains a strictly positive constant bias equal to the smallest positive binary64 value: for example, node $(1, 6)$ has $h_{1,6} = 2^{-1074} > 0$ and always chooses right. Rounding it to zero would reverse that decision. A compiler can instead fold such a constant node to a Boolean branch choice; it need not numerically represent that bias in a fixed-point dot product. This is different from arbitrarily pruning a nonconstant tiny-weight split.

Aggressive L1 can remove needed directions or leaf features; SFR cannot bring them back. Moderate penalties can retain useful support while imposing excessive shrinkage, in which case refitting helps. The empirical scan tests both effects; it does not assume that stronger sparsity is always preferable. The training objective is cross-entropy, whereas model selection uses low-background acceptance. A lower training loss can therefore yield a worse selected-working- point efficiency, motivating exact saved-checkpoint selection and an honest separation of terminal and best validation iterates.

6 Comparison with XGBoost and current results

6.1 Reference configuration and metric

XGBoost fits an additive ensemble of axis-aligned trees with scalar constant leaves, using gradient/Hessian information for greedy tree construction and regularized leaf updates [6]. Our reference uses `xgboost==3.2.0`, 100 estimators, maximum depth 4, learning rate 0.1, row and feature subsampling 0.8 each, `tree_method='hist'`, `objective='binary:logistic'`, `n_jobs=1` and random state 42. The configuration is `configs/bdt_qg_vs_wzt_seed42.yaml`; the trainer is `src/train_bdt.py`. There is no new XGBoost tuning or refit in this note. Settings not explicitly overridden are those of the pinned XGBoost version; the native JSON in the handover fixes the actual trained reference.

Its actual structure has 1,499 internal thresholds, 1,599 leaf values and one base bias, giving 3,099 scalar slots. Feature indices and topology are omitted from this count, just as metadata are omitted from the oblique coefficient count.

The primary metric is signal efficiency at background efficiency 0.01, interpolated on the empirical ROC by the common evaluation implementation in `src/evaluate.py`. AUC is secondary. Separately, each exported JSON records a realizable strict margin threshold with at most $\lfloor 0.01 \cdot 52{,}511 \rfloor = 525$ accepted validation background events. This deterministic operating point and an interpolated ROC value are distinct concepts. Raw and standardized exports each carry a separately calibrated threshold; firmware must use the threshold for its chosen input space.

6.2 Pinned v3 models

Table 2: Frozen implementation comparison on the same validation partition. All oblique models have linear leaves; the XGBoost leaves are constant. Dense slots and exact nonzeros include biases. CPU is frozen bulk time per event in microseconds, not FPGA or single-event latency. Compact refit is the explicit default, irrespective of which row has the highest efficiency.

Model	Layout	Dense	Nonzero	ϵ_s [%]	AUC	CPU [μs]
Compact refit	3 × D4	1581	418	56.20	0.92733	0.974
Higher efficiency	3 × D4	1581	837	57.12	0.92967	1.131
One-tree reference	1 × D4	527	452	55.63	0.92704	0.412
XGBoost	100 × D4	3099	—	55.48	0.92056	1.083

The Compact refit uses about half the nonzeros of the higher-efficiency alternative (418 versus 837) and 49% fewer dense slots than XGBoost (1,581 versus 3,099). These are different comparisons: 418 nonzeros are not an array of only 418 allocated slots, and a threshold in XGBoost does not have the same hardware cost as an oblique multiply-add. The one-tree model is slightly larger in nonzero count than Compact refit, but has fewer paths/trees and a smaller sparse arithmetic bound.

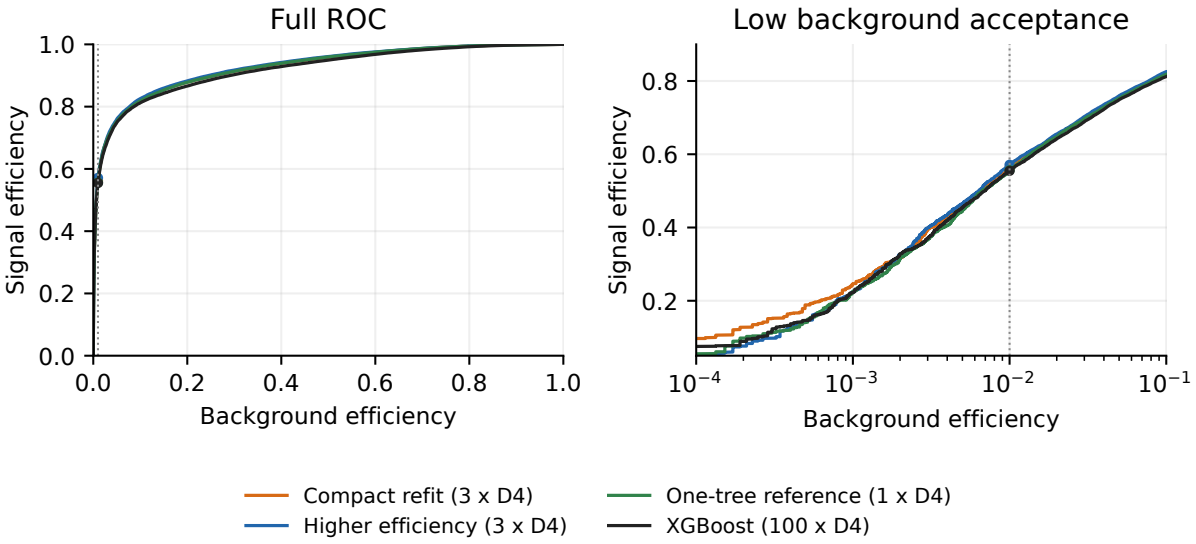


Figure 1: ROCs regenerated from the exact frozen saved models on validation. The right panel resolves the low-background region. Dots mark the interpolated 1% background working point. These curves do not include the held 57.24% full-stage diagnostic or a newly selected running model.

Table 3: Analytical sparse selected-path bounds for the binary standardized models, from `sparsity_profile`. Reachable coefficients omit only algebraically unreachable branches. All-node counts sum feature terms over all reachable split and leaf functions; path counts bound a selected route. Bias additions are not multiplications. A dense three-tree linear forest has 240 selected-path or 1,488 all-node multiplications and 12 path decisions.

Model	Reachable	Path mult.	All-node mult.	Path decisions
Compact refit	379	80	327	12
Higher efficiency	837	163	744	12
One-tree reference	452	65	421	4

6.3 The fixed-support penalty study

Table 4 keeps each parent and its refits on the same selected support. The scan includes mild and strong split-only or joint penalties and a smaller one-tree architecture. The Compact refit row recovers about 1.61 percentage points from its terminal L1 parent with no added coefficients. The higher-efficiency leaf-only refit improves its own parent by about 0.21 points; that small gain must not be overstated.

Table 4: Ten-parent SFR study. Efficiencies are percentages at 1% background. All rows are three D4 linear trees except $*$ (one D4 tree). Full terminal and best saved full are deliberately separate. † marks a held full-stage diagnostic, excluded from implementation selection; a completed leaf stage remains eligible. `LPtimeout` is inconclusive, not a proof of separation.

Selector	λ_s/λ_ℓ	Support	Parent	Leaf	Full terminal	Best full	Full state
both10	`0.0001/0.0001`	520	55.58	55.82	56.49	56.49	complete
split30	`0.0003/1e-5`	675	56.98	56.74	—	56.71†	LP timeout
leaf3	`1e-5/3e-5`	837	56.91	57.12	—	57.24†	LP timeout
both30	`0.0003/0.0003`	394	54.48	54.85	—	54.78†	separated
compact1t*	`1e-5/1e-5`	403	55.20	55.06	55.02	55.08	complete
split100	`0.001/1e-5`	561	56.06	56.32	56.28	56.46	complete
both3	`3e-5/3e-5`	777	56.62	56.75	56.66	56.66	complete
control	`1e-5/1e-5`	889	56.40	56.45	56.72	56.74	complete
split300	`0.003/1e-5`	418	54.59	54.81	55.98	56.20	complete
both100	`0.001/0.001`	317	51.27	52.52	—	52.94†	separated

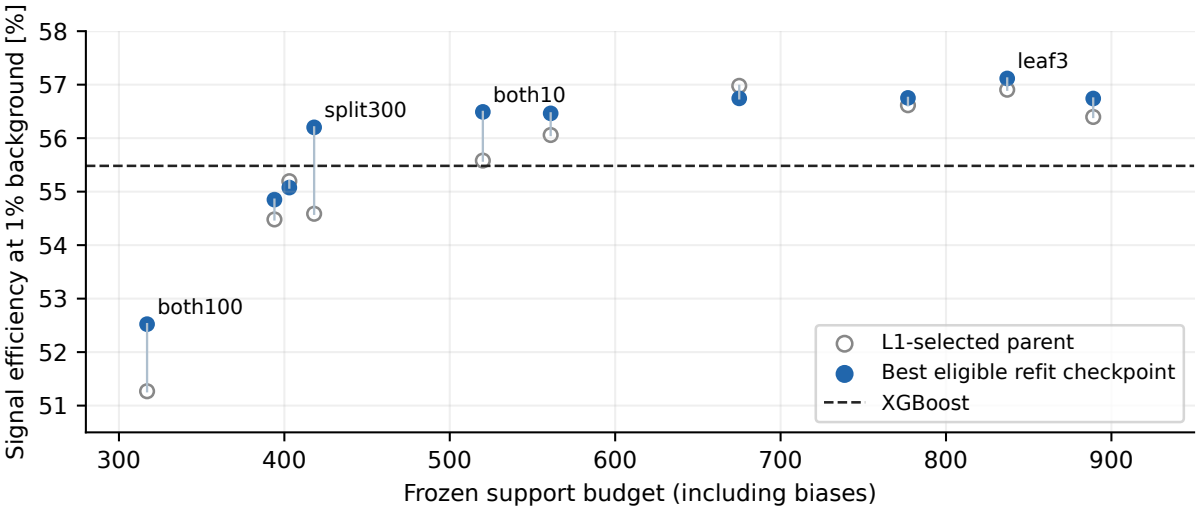


Figure 2: Each vertical segment connects an L1-selected terminal parent with its best eligible leaf/full refit checkpoint at the same support budget. Held full stages are excluded, but their converged leaf-only results can contribute. This figure illustrates the separation of support selection from coefficient shrinkage; no support regrowth is used.

6.4 Uncertainty, running studies and timing limits

The frozen comparison includes a paired class-stratified bootstrap with 1,000 replicates, seed 20261025, using the same interpolated working point in each replicate. Table 5 gives percentile intervals for differences against XGBoost. These intervals condition on the fitted/selected models. They do not account for hyperparameter/checkpoint selection or training-seed variation. The code is `src/tao/uncertainty.py`; ties and resampled zero-weight rows are handled consistently with weighted ROC evaluation.

Table 5: Conditional validation difference versus XGBoost, in percentage points. A positive point estimate is not by itself evidence of an established improvement.

Model	Difference [pp]	Paired 95% interval [pp]
Compact refit	+0.72	[-0.43, +1.52]
Higher efficiency	+1.63	[+0.65, +2.55]
One-tree reference	+0.15	[-1.08, +0.97]

In particular, Compact refit's interval includes zero. It is an attractive compression point with a slightly higher validation point estimate, not a confirmed held-out superiority claim. The higher-efficiency alternative has an interval above zero in this conditional comparison, but still needs an independent final evaluation if a generalization claim is to be made.

The dated research snapshot on 26 October also contains an unfinished five-tree D4 linear forest at 57.13%, AUC 0.92907, with 2,635 dense slots and 1,258 nonzero coefficients. Its score is only about 0.014 percentage points above the frozen 837-coefficient alternative; it is not converged and is not promoted. The exact inspected checkpoint, configuration and status are in `docs/handover_status_2026-10-26.json`. This research observation is kept separate from Table 2, rather than replacing a frozen row with a transient training value.

The CPU values in Table 2 use a full batch of 132,800 validation events, one thread, nine timing repeats and the median wall time per event. They compare the current NumPy TAO API with compiled XGBoost. The NumPy `dense` traversal can evaluate a node's dot product over a full input

batch before selecting reaching rows; it is not executing only the sparse path arithmetic of Table 3. There is no hidden claim of an equally optimized CPU implementation, batch-size-one trigger latency, or an FPGA speedup. FPGA cost additionally depends on feature distribution/fanout, coefficient formats, adder trees, multiplier reuse, memory layout, pipelining and overflow policy.

7 Reproduction, code map and the firmware handover

7.1 Pinned environment and entry points

Training and input regeneration use Python 3.10 with the pinned `requirements.txt`: NumPy 1.26.4, SciPy 1.15.2, scikit-learn 1.7.2, XGBoost 3.2.0, PyYAML 6.0.3 and matplotlib 3.10.7. The reference CLIP checkout is `/users/robert.schoefbeck/CMS/tree_benchmark`; its Conda environment is `tree-benchmark`. On CLIP use the conda-forge CPU XGBoost build, since the PyPI Linux wheel requires newer glibc than the reference login environment. No GPU is needed.

```
source /software/f2022/software/mamba/24.3.0-0/etc/profile.d/conda.sh
conda activate tree-benchmark
cd /users/robert.schoefbeck/CMS/tree_benchmark
export PYTHONPATH="$PWD/src:$PWD" PYTHONDONTWRITEBYTECODE=1
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1
python scripts/print_tao_tree.py \
  --model fpga_handoff/v3/compact_refit_3t_d4/standardized/model.json \
  --precision 17 --output /tmp/compact_forest.txt
```

For a new training account or checkout, the complete environment-creation and controlled refit recipe is in `docs/DORIAN_HANDOVER.md`. The datasets and pickle archives are deliberately not in Git. Exact reproduction of the refit starts from the hash-pinned terminal parent; rebuilding its entire L1 path can produce differently serialized checkpoint bytes and requires a new explicit source hash/run name. Existing named runs must not be resumed under changed training code or stopping policies. The standalone `paper/` folder, however, contains all generated text/tables and figures needed to compile this PDF without datasets, model pickles or a Python training environment.

Table 6: Implementation map. The function names identify the code implementing the mathematical objects, rather than a generic TAO library API.

Object / operation	Entry point
Routing, logits, objective	`HardTree`; `HardTAOClassifier` in `src/tao/hard.py`
Weighted leaf optimization	`fit_leaf_parameters`; `ce_value_gradient`
Joint leaf design/refit	`_joint_design`; `global_refit`
Reduced split proposals	`_hard_split_update`; `FrozenSupportTAO._split_update`
Sorted hard threshold search	`best_threshold` in `src/tao/split_search.py`
Node-resumable epoch	`begin_epoch`; `step_epoch` in `src/tao/epoch.py`
Masks and separation	`freeze_support`; `continue_full_refit`; `separation_check`
Stopping and exact-zero work counts	`leaf_kkt_residual`; `convergence_check`; `sparsity_profile`
Read-only equations	`TreePrinter.print_tree` in `src/tao/printing.py`
Portable inference / firmware export	`export_model`; `predict_margin`; `cpp_header` in `src/tao/portable.py`

7.2 Frozen coefficients and reference verification

`fpga_handoff/v3/manifest.json` hashes the candidate JSONs, hexadecimal C++ headers, feature contract, conservative training-box ranges and golden vectors. Each candidate has both standardized

and raw-input variants; choose one and use its matching vectors and working-point threshold. The default JSON hash is

`0ce4fabc4ca74d3fcd2ddb8c94bb6b614b4f2fb998a9225873e167977922d62a`

The corresponding immutable pickle is named by the checkpoint hash given in Section 5.2, under `models/support_refit_scan_v1_selected/` on CLIP. The binary portable JSON and headers suffice for firmware; Python pickles are not needed there.

The C++17 reference in `fpga_handoff/reference.cpp` passed all six v3 variants: 1,594 vector evaluations with exact leaf assignments and maximum margin difference below 2.5×10^{-14}. The verifier `scripts/verify_fpga_handoff.py` needs only Python's standard library and a C++17 compiler. It checks bundle hashes before compiling/running the golden vectors and preserves an existing frozen verification report. Fixed-point verification remains a separate next step. v1 and v2 are retained historical bundles; `fpga_handoff/CURRENT_MODEL.json` selects v3 Compact refit for new implementation work.

Practical next measurements. The next implementation study should establish explicit feature/coefficient/ accumulator formats, evaluate quantization and near-boundary routing changes, recalibrate a realizable validation working point, and measure synthesis resources and latency at a specified device and clock. Training improvements worth a controlled follow-up include better-conditioned auxiliary solves, removing frozen columns from the joint design, and generated sparse inference. They are not silently assumed in the present accuracy or CPU comparison. Further test evaluation should wait until the model-selection and implementation choices are settled; the historical XGBoost-only check in Section 1 must remain disclosed.

8 Conclusions and implementation priorities

The present result is a compact floating-point classifier at the selected low-background working point. The default three-tree depth-four model uses linear leaves and 418 exact nonzero coefficients, compared with 837 for the higher-efficiency oblique alternative and 3,099 scalar slots in the XGBoost reference. Its validation efficiency is 56.20%, versus 55.48% for XGBoost. The uncertainty interval on that difference includes zero. The justified claim is therefore competitive validation performance with fewer stored nonzero coefficients, rather than established superior generalization or a measured FPGA speedup.

The training study separates two effects that should not be conflated: selecting a useful set of coefficients and shrinking their fitted values. For the Compact support, Select–Freeze–Refit raises the terminal penalized parent's validation efficiency from 54.59% to a best eligible saved value of 56.20% without adding coefficients. Other supports respond differently, and some unpenalized stages are held for separation or an inconclusive diagnostic. The procedure is consequently a controlled empirical strategy, not a guarantee that removing regularization always improves a tree. Likewise, a numerically converged source run and a validation-selected earlier checkpoint are different objects; both are identified in the handover.

The immediate implementation target remains the frozen v3 Compact refit. First preserve its feature order, margin sign, strict cut convention and constant routes in the firmware representation. Next choose explicit numeric formats and measure the effect of quantization on routes and on signal efficiency at a realizable background threshold. Finally synthesize for a specified FPGA and clock, reporting resources, latency and initiation interval alongside the classifier metric. These measurements will determine whether the reduction in coefficient count translates into the hardware advantage motivating this study. Any later compression or training improvement should be compared against the same frozen model and data contract, with changed coefficients and working points exported as a new version.

References

[1] HEPHY Analysis Software. Oblique tree benchmark, 2026. URL https://github.com/HephyAnalysisSW/oblique-tree-benchmark. Implementation, frozen v3 FPGA handover and dated research records; exact source fingerprints accompany this note.

[2] OpenML. hls4ml_lhc_jets_hlf, dataset 42468. Dataset catalogue. URL https://www.openml.org/d/42468. The project uses its archived feature arrays and fixed splits, rather than a new split from the catalogue.

[3] Miguel Á. Carreira-Perpiñán and Pooya Tavallali. Alternating optimization of decision trees, with application to learning sparse oblique trees. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/hash/185c29dc24325934ee377cfda20e414c-Abstract.html.

[4] Suryabhan Singh Hada, Miguel Á. Carreira-Perpiñán, and Arman Zharmagambetov. Sparse oblique decision trees: A tool to understand and manipulate neural net features. arXiv:2104.02922, 2021. URL https://arxiv.org/abs/2104.02922. First posted in 2021; revised manuscript and journal version in 2023.

[5] Nicolai Meinshausen. Relaxed lasso. *Computational Statistics & Data Analysis*, 52(1):374–393, 2007. doi: 10.1016/j.csda.2006.12.019. URL https://people.math.ethz.ch/~nicolai/relaxo.pdf.

[6] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785. URL https://arxiv.org/abs/1603.02754.

A Input feature order and model identity

All equations use the following exact standardized feature names. Their raw counterparts have the same order; the shared scaler is in `fpga_handoff/v3/input_contract.json`.

Index	Feature
0	`zlogz`
1	`c1_b0_mmdt`
2	`c1_b1_mmdt`
3	`c1_b2_mmdt`
4	`c2_b1_mmdt`
5	`c2_b2_mmdt`
6	`d2_b1_mmdt`
7	`d2_b2_mmdt`
8	`d2_a1_b1_mmdt`
9	`d2_a1_b2_mmdt`
10	`m2_b1_mmdt`
11	`m2_b2_mmdt`
12	`n2_b1_mmdt`
13	`n2_b2_mmdt`
14	`mass_mmdt`
15	`multiplicity`

The repository base examined for this note is

`9e08df2b7e2e99d34203544c344f3fdfc775f5a8`

The added read-only printer and input generator are fingerprinted separately in `sources/provenance.json`, together with the unchanged numerical training files, selected model, data, generated tables and figures. Model selection is defined by the frozen source hashes, not by whatever checkpoint happens to be latest when this PDF is opened.

B Complete Compact refit forest printout

This record is generated by `TreePrinter.format_tree` from the exact v3 standardized model. It lists every internal decision and every leaf output in preorder, with heap indices and root-to-node paths. A node labelled unreachable is excluded only by an input-independent constant ancestor, not by absent training or validation examples. Numeric values are displayed with nine significant digits. The full 17-digit record is checked in as `generated/compact_forest_full_precision.txt`. These listings are for inspection; implementation should consume the frozen JSON or hexadecimal header. In all trees, a leaf output contributes to the background logit; the signal margin is the *negative sum* of the three selected outputs.

B.1 Tree 0

```
TREE 0: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).
N0 [path=root; heap=0; reachable]
  h_0_0(x) = 1 * c1_b2_mmdt + 0.421620399
  <= 0 -> N1; > 0 -> N2
  N1 [path=L; heap=1; reachable]
    h_0_1(x) = 0
    <= 0 -> N3; > 0 -> N4; CONSTANT: always LEFT
    N3 [path=LL; heap=3; reachable]
      h_0_3(x) = 0
      <= 0 -> N7; > 0 -> N8; CONSTANT: always LEFT
      N7 [path=LLL; heap=7; reachable]
        h_0_7(x) = 0.318308228 * c1_b0_mmdt + 0.330704464
        <= 0 -> L0; > 0 -> L1
        L0 [path=LLLL; heap=15; reachable]
          f_0_0(x) = 0.723115375 * zlogz - 0.424343964 * c1_b0_mmdt - 0.446947898 * c1_b1_mmdt - 0.415001941
↪ * c2_b1_mmdt - 0.139216682 * d2_b1_mmdt - 0.139216682 * d2_a1_b1_mmdt + 0.54944101 * m2_b1_mmdt -
↪ 0.495073946 * m2_b2_mmdt + 0.059415836 * n2_b1_mmdt + 0.858013396 * multiplicity - 1.35727329
        L1 [path=LLLR; heap=16; reachable]
          f_0_1(x) = 1.51929697 * zlogz + 0.669978191 * c1_b0_mmdt - 3.56315299 * c1_b2_mmdt + 0.613457803 *
↪ c2_b1_mmdt - 0.396278128 * d2_b1_mmdt + 0.152304141 * d2_b2_mmdt - 0.396278128 * d2_a1_b1_mmdt -
↪ 0.199974866 * m2_b2_mmdt + 0.980332415 * n2_b1_mmdt - 0.420497004 * n2_b2_mmdt + 0.396719294 * mass_mmdt +
↪  1.34144785 * multiplicity - 1.72085229
      N8 [path=LLR; heap=8; UNREACHABLE (constant ancestor)]
        h_0_8(x) = 0
        <= 0 -> L2; > 0 -> L3; CONSTANT: always LEFT
        L2 [path=LLRL; heap=17; UNREACHABLE (constant ancestor)]
          f_0_2(x) = 0
        L3 [path=LLRR; heap=18; UNREACHABLE (constant ancestor)]
          f_0_3(x) = 0
    N4 [path=LR; heap=4; UNREACHABLE (constant ancestor)]
      h_0_4(x) = 0
      <= 0 -> N9; > 0 -> N10; CONSTANT: always LEFT
      N9 [path=LRL; heap=9; UNREACHABLE (constant ancestor)]
        h_0_9(x) = -0.0491155824 * c1_b0_mmdt + 0.000379329885
        <= 0 -> L4; > 0 -> L5
        L4 [path=LRLL; heap=19; UNREACHABLE (constant ancestor)]
          f_0_4(x) = 0
        L5 [path=LRLR; heap=20; UNREACHABLE (constant ancestor)]
          f_0_5(x) = 0
      N10 [path=LRR; heap=10; UNREACHABLE (constant ancestor)]
        h_0_10(x) = 1.75931373e-09 * c1_b0_mmdt - 3.92622475e-09 * m2_b2_mmdt + 1.22858604e-07 * multiplicity
↪  - 1.097723e-07
        <= 0 -> L6; > 0 -> L7
        L6 [path=LRRL; heap=21; UNREACHABLE (constant ancestor)]
          f_0_6(x) = 0
        L7 [path=LRRR; heap=22; UNREACHABLE (constant ancestor)]
          f_0_7(x) = 0
  N2 [path=R; heap=2; reachable]
    h_0_2(x) = 0.0881065951 * c1_b2_mmdt + 0.0025365055 * c2_b2_mmdt + 0.17735909 * m2_b1_mmdt + 0.0312233619
↪  * mass_mmdt + 0.100640807 * multiplicity + 0.0190586642
    <= 0 -> N5; > 0 -> N6
    N5 [path=RL; heap=5; reachable]
      h_0_5(x) = 0
      <= 0 -> N11; > 0 -> N12; CONSTANT: always LEFT
      N11 [path=RLL; heap=11; reachable]
        h_0_11(x) = -9.45167785e-16 * d2_b1_mmdt - 5.84426162e-16
        <= 0 -> L8; > 0 -> L9
        L8 [path=RLLL; heap=23; reachable]
          f_0_8(x) = 0.472450802 * zlogz + 0.0989313467 * c1_b0_mmdt - 0.831430736 * c1_b2_mmdt - 0.431555721
↪  * c2_b2_mmdt + 0.0150358961 * d2_b1_mmdt + 0.648844514 * d2_b2_mmdt + 0.0150358961 * d2_a1_b1_mmdt -
↪ 3.09641516 * d2_a1_b2_mmdt + 1.49849996 * m2_b1_mmdt + 1.81153643 * n2_b1_mmdt - 0.893777123 * n2_b2_mmdt
↪ + 0.182450792 * multiplicity - 2.83938374
```

```
        L9 [path=RLLR; heap=24; reachable]
          f_0_9(x) = 0.232327574 * zlogz - 0.23205911 * c1_b0_mmdt - 0.516471161 * c1_b1_mmdt - 0.848527306 *
↪  c2_b1_mmdt - 1.97477014 * c2_b2_mmdt + 1.42114491 * d2_b2_mmdt + 0.536602443 * m2_b1_mmdt + 0.651756707 *
↪  mass_mmdt - 1.64666395
      N12 [path=RLR; heap=12; UNREACHABLE (constant ancestor)]
        h_0_12(x) = 9.89173599e-11 * c1_b2_mmdt + 1.79437153e-09 * c2_b1_mmdt + 1.81919634e-09 * c2_b2_mmdt +
↪  1.04854961e-09
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; UNREACHABLE (constant ancestor)]
          f_0_10(x) = 0
        L11 [path=RLRR; heap=26; UNREACHABLE (constant ancestor)]
          f_0_11(x) = 0
    N6 [path=RR; heap=6; reachable]
      h_0_6(x) = 0.164748862 * c1_b2_mmdt + 0.185195199 * c2_b2_mmdt + 0.271888733 * d2_a1_b2_mmdt +
↪ 1.06018366 * mass_mmdt - 0.265110937 * multiplicity - 1.32289565
      <= 0 -> N13; > 0 -> N14
      N13 [path=RRL; heap=13; reachable]
        h_0_13(x) = 0.635416434 * c1_b2_mmdt - 0.475835509 * multiplicity + 0.00561843162
        <= 0 -> L12; > 0 -> L13
        L12 [path=RRLL; heap=27; reachable]
          f_0_12(x) = 0.000756449729 * zlogz + 0.576353531 * c1_b0_mmdt + 1.62064775 * c1_b1_mmdt -
↪ 2.48681119 * c1_b2_mmdt - 2.10518311 * c2_b1_mmdt + 1.37970957 * c2_b2_mmdt - 0.0135562553 * d2_b1_mmdt +
↪ 0.338362995 * d2_b2_mmdt - 0.0135562553 * d2_a1_b1_mmdt - 1.72149735 * d2_a1_b2_mmdt - 0.38025814 *
↪ m2_b1_mmdt - 0.351321056 * m2_b2_mmdt + 2.69042188 * n2_b1_mmdt - 0.919390109 * n2_b2_mmdt + 0.32251432 *
↪ mass_mmdt + 0.582538651 * multiplicity - 3.45473882
        L13 [path=RRLR; heap=28; reachable]
          f_0_13(x) = -0.140117186 * c1_b2_mmdt + 0.652505422 * c2_b1_mmdt + 0.253434919 * d2_b1_mmdt -
↪ 0.602112106 * d2_b2_mmdt + 0.253434919 * d2_a1_b1_mmdt - 0.527210682 * d2_a1_b2_mmdt - 0.00193453091 *
↪ n2_b2_mmdt - 1.64156297 * mass_mmdt + 0.0192123329 * multiplicity - 2.68453555
      N14 [path=RRR; heap=14; reachable]
        h_0_14(x) = 0
        <= 0 -> L14; > 0 -> L15; CONSTANT: always LEFT
        L14 [path=RRRL; heap=29; reachable]
          f_0_14(x) = -0.727395772 * zlogz - 1.0021119 * c1_b0_mmdt - 1.77913852 * c1_b1_mmdt + 1.29863387 *
↪ c1_b2_mmdt - 5.22419531 * c2_b1_mmdt + 1.67449112 * c2_b2_mmdt + 0.609159654 * d2_b1_mmdt - 1.14240277 *
↪ d2_b2_mmdt + 0.609159654 * d2_a1_b1_mmdt + 0.462711749 * d2_a1_b2_mmdt - 0.379623445 * m2_b1_mmdt -
↪ 1.49230987 * m2_b2_mmdt + 1.44153754 * n2_b1_mmdt - 4.67940637 * mass_mmdt + 1.29820647 * multiplicity +
↪ 5.68806483
        L15 [path=RRRR; heap=30; UNREACHABLE (constant ancestor)]
          f_0_15(x) = 0
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```

B.2 Tree 1

```
TREE 1: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).
N0 [path=root; heap=0; reachable]
  h_1_0(x) = 1 * c1_b2_mmdt + 0.421515048
  <= 0 -> N1; > 0 -> N2
  N1 [path=L; heap=1; reachable]
    h_1_1(x) = -2.3776136e-11 * multiplicity - 3.17032814e-11
    <= 0 -> N3; > 0 -> N4
    N3 [path=LL; heap=3; reachable]
      h_1_3(x) = -0.0032891119 * zlogz + 0.0142175742 * d2_b1_mmdt + 0.00121827303 * d2_a1_b1_mmdt -
↪ 0.000244599886 * m2_b2_mmdt - 0.00553888952 * n2_b2_mmdt + 0.295132576 * mass_mmdt + 0.0218087714
      <= 0 -> N7; > 0 -> N8
      N7 [path=LLL; heap=7; reachable]
        h_1_7(x) = 0.0213173363 * zlogz + 0.612901764 * c1_b0_mmdt - 0.0606512036
        <= 0 -> L0; > 0 -> L1
        L0 [path=LLLL; heap=15; reachable]
          f_1_0(x) = 1.02493155 * zlogz + 0.475780074 * c1_b0_mmdt - 3.46865641 * c1_b1_mmdt + 1.39660226 *
↪ c1_b2_mmdt + 0.0808524935 * d2_b1_mmdt + 0.0625866107 * d2_b2_mmdt + 0.0808524935 * d2_a1_b1_mmdt +
↪ 0.265262813 * d2_a1_b2_mmdt + 0.0469595498 * m2_b1_mmdt - 0.00734786374 * m2_b2_mmdt + 3.49945192 *
↪ mass_mmdt - 0.346146245
        L1 [path=LLLR; heap=16; reachable]
          f_1_1(x) = 2.42880675 * zlogz + 3.52193453 * c1_b0_mmdt - 2.27198818 * c1_b1_mmdt - 0.640806627 *
↪ c1_b2_mmdt + 0.0986683506 * d2_b1_mmdt + 0.101873458 * d2_b2_mmdt + 0.0986683506 * d2_a1_b1_mmdt +
↪ 0.331194728 * m2_b1_mmdt + 0.0539714566 * n2_b1_mmdt + 3.24454093 * mass_mmdt + 0.503474228 * multiplicity
↪  - 1.50898053
      N8 [path=LLR; heap=8; reachable]
        h_1_8(x) = 7.32169007e-19 * zlogz - 2.06778204e-19 * d2_b1_mmdt - 1.91895639e-18 * d2_a1_b1_mmdt +
↪ 4.16333634e-17 * mass_mmdt + 5.02329003e-19
        <= 0 -> L2; > 0 -> L3
        L2 [path=LLRL; heap=17; reachable]
          f_1_2(x) = 0.398116558 * d2_b1_mmdt + 0.398116558 * d2_a1_b1_mmdt - 0.660830844 * d2_a1_b2_mmdt -
↪ 0.114827421 * m2_b2_mmdt - 0.310979867 * n2_b2_mmdt + 0.846928396 * multiplicity - 0.9437156
        L3 [path=LLRR; heap=18; reachable]
          f_1_3(x) = 0.244294729 * zlogz + 0.880510729 * d2_b2_mmdt - 1.42081066 * m2_b1_mmdt + 1.71265082 *
↪ multiplicity - 2.14718376
    N4 [path=LR; heap=4; reachable]
      h_1_4(x) = 0
      <= 0 -> N9; > 0 -> N10; CONSTANT: always LEFT
      N9 [path=LRL; heap=9; reachable]
        h_1_9(x) = 0.128095519 * m2_b1_mmdt + 0.00063403479 * multiplicity - 0.0832689733
        <= 0 -> L4; > 0 -> L5
        L4 [path=LRLL; heap=19; reachable]
          f_1_4(x) = 0.232595756 * zlogz + 0.336338146 * c1_b0_mmdt + 0.186230458 * d2_b1_mmdt + 0.186230458
↪ * d2_a1_b1_mmdt - 0.0975771333 * m2_b1_mmdt + 0.100432843 * m2_b2_mmdt - 0.712895383 * multiplicity -
↪ 1.51109101
        L5 [path=LRLR; heap=20; reachable]
          f_1_5(x) = 0.26497515 * d2_b2_mmdt + 0.322656527 * n2_b2_mmdt - 0.656358636
      N10 [path=LRR; heap=10; UNREACHABLE (constant ancestor)]
        h_1_10(x) = 3.11856008e-17 * zlogz + 1.03393415e-17 * d2_b1_mmdt + 3.62950285e-17 * d2_a1_b1_mmdt -
↪ 3.214346e-17 * m2_b1_mmdt - 5.71317478e-17 * m2_b2_mmdt + 5.83355278e-17 * mass_mmdt + 1.70316265e-11 *
↪ multiplicity - 1.89719708e-11
        <= 0 -> L6; > 0 -> L7
        L6 [path=LRRL; heap=21; UNREACHABLE (constant ancestor)]
          f_1_6(x) = 0
        L7 [path=LRRR; heap=22; UNREACHABLE (constant ancestor)]
          f_1_7(x) = 0
  N2 [path=R; heap=2; reachable]
    h_1_2(x) = 1.08273826 * c1_b2_mmdt + 0.24935691 * c2_b2_mmdt - 0.0971598576 * d2_b1_mmdt + 0.256354852 *
↪ d2_b2_mmdt - 0.0929252735 * d2_a1_b1_mmdt - 0.228292085 * m2_b1_mmdt + 0.0567631958 * mass_mmdt +
↪ 0.28027011 * multiplicity - 0.13138567
    <= 0 -> N5; > 0 -> N6
    N5 [path=RL; heap=5; reachable]
      h_1_5(x) = 0
      <= 0 -> N11; > 0 -> N12; CONSTANT: always LEFT
```

```
      N11 [path=RLL; heap=11; reachable]
        h_1_11(x) = -0.259890766 * d2_a1_b2_mmdt + 0.208572797 * m2_b1_mmdt + 3.31046124 * mass_mmdt -
↪ 0.0387863599
        <= 0 -> L8; > 0 -> L9
        L8 [path=RLLL; heap=23; reachable]
          f_1_8(x) = 1.49591861 * zlogz + 0.441575961 * c1_b0_mmdt + 1.71623752 * c1_b2_mmdt + 0.707286758 *
↪ d2_a1_b2_mmdt + 0.136799401 * n2_b1_mmdt - 0.0922364294 * n2_b2_mmdt - 4.1902612 * mass_mmdt + 0.994771372
↪  * multiplicity + 0.902352018
        L9 [path=RLLR; heap=24; reachable]
          f_1_9(x) = 1.06616852 * zlogz + 0.26139261 * c1_b0_mmdt - 1.59073627 * c1_b1_mmdt - 3.31259981 *
↪ c2_b1_mmdt + 2.90835606 * c2_b2_mmdt + 0.0402529973 * d2_b1_mmdt + 0.0402529973 * d2_a1_b1_mmdt + 3.30625
↪ * d2_a1_b2_mmdt - 2.78491928 * m2_b1_mmdt + 0.342727361 * m2_b2_mmdt - 0.69660259 * n2_b2_mmdt +
↪ 1.00497391 * mass_mmdt + 2.54220983 * multiplicity + 0.0829818883
      N12 [path=RLR; heap=12; UNREACHABLE (constant ancestor)]
        h_1_12(x) = 3.32650029e-05 * c1_b2_mmdt + 1.11948505e-05 * c2_b2_mmdt + 2.49204468e-05 *
↪ d2_a1_b2_mmdt + 1.99509567e-06 * m2_b2_mmdt + 0.000137262248 * n2_b1_mmdt + 9.50083532e-05
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; UNREACHABLE (constant ancestor)]
          f_1_10(x) = 0
        L11 [path=RLRR; heap=26; UNREACHABLE (constant ancestor)]
          f_1_11(x) = 0
    N6 [path=RR; heap=6; reachable]
      h_1_6(x) = 4.94065646e-324
      <= 0 -> N13; > 0 -> N14; CONSTANT: always RIGHT
      N13 [path=RRL; heap=13; UNREACHABLE (constant ancestor)]
        h_1_13(x) = -2.68948213e-10 * zlogz - 5.82051432e-11 * c2_b1_mmdt - 2.85373406e-10 * d2_b1_mmdt +
↪ 2.05721037e-10 * d2_b2_mmdt - 1.06470833e-10 * d2_a1_b1_mmdt + 9.05405039e-12 * d2_a1_b2_mmdt + 3.42554664
↪ e-10 * m2_b2_mmdt + 2.13357549e-10 * n2_b2_mmdt - 8.89638165e-10 * mass_mmdt + 1.4924407e-09 *
↪ multiplicity - 1.72964941e-09
        <= 0 -> L12; > 0 -> L13
        L12 [path=RRLL; heap=27; UNREACHABLE (constant ancestor)]
          f_1_12(x) = 0
        L13 [path=RRLR; heap=28; UNREACHABLE (constant ancestor)]
          f_1_13(x) = 0
      N14 [path=RRR; heap=14; reachable]
        h_1_14(x) = -0.0357752902 * c1_b2_mmdt + 1.00738533 * mass_mmdt - 1.85526559
        <= 0 -> L14; > 0 -> L15
        L14 [path=RRRL; heap=29; reachable]
          f_1_14(x) = 0.992368325 * zlogz - 4.80052499 * c1_b1_mmdt + 2.90036981 * c1_b2_mmdt - 0.714839331 *
↪  c2_b1_mmdt - 0.349819173 * d2_b1_mmdt - 0.0440891415 * d2_b2_mmdt - 0.349819173 * d2_a1_b1_mmdt +
↪ 0.777214702 * d2_a1_b2_mmdt + 0.176693713 * m2_b1_mmdt + 2.01859804 * n2_b1_mmdt - 1.16458901 * n2_b2_mmdt
↪  - 1.53336443 * mass_mmdt + 0.283339646 * multiplicity + 2.63807278
        L15 [path=RRRR; heap=30; reachable]
          f_1_15(x) = -0.485616316 * c1_b0_mmdt - 0.625011056 * c2_b2_mmdt + 0.51939675 * d2_b1_mmdt +
↪ 0.199901858 * d2_b2_mmdt + 0.51939675 * d2_a1_b1_mmdt - 0.491209214 * m2_b2_mmdt + 2.38441955 * mass_mmdt -
↪  0.280424752 * multiplicity - 5.0660948
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```

B.3 Tree 2

```
TREE 2: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).

N0 [path=root; heap=0; reachable]
  h_2_0(x) = 0.451963069 * c1_b2_mmdt - 0.00922949319 * m2_b1_mmdt + 0.315856319 * mass_mmdt + 0.0990362496
  <= 0 -> N1; > 0 -> N2

  N1 [path=L; heap=1; reachable]
    h_2_1(x) = -0.0393006101 * zlogz + 0.0190956747 * multiplicity + 0.00401865025
    <= 0 -> N3; > 0 -> N4

    N3 [path=LL; heap=3; reachable]
      h_2_3(x) = 0
      <= 0 -> N7; > 0 -> N8; CONSTANT: always LEFT

      N7 [path=LLL; heap=7; reachable]
        h_2_7(x) = -0.338370941 * zlogz - 0.197456658 * c1_b0_mmdt + 0.505337081 * multiplicity + 0.323631119
        <= 0 -> L0; > 0 -> L1

        L0 [path=LLLL; heap=15; reachable]
          f_2_0(x) = 0.391007555 * c1_b0_mmdt + 2.7854501 * c1_b1_mmdt - 5.98969282 * c1_b2_mmdt +
↪ 0.733907917 * c2_b1_mmdt - 0.0568167023 * d2_b2_mmdt - 0.461051796 * m2_b1_mmdt + 0.358625148 * m2_b2_mmdt
↪  + 0.00378192334 * n2_b2_mmdt - 2.04581277 * mass_mmdt - 0.432532063 * multiplicity - 1.93961875

        L1 [path=LLLR; heap=16; reachable]
          f_2_1(x) = -0.441179138 * zlogz - 3.22570484 * c1_b2_mmdt - 0.0681594455 * d2_b1_mmdt -
↪ 0.0681594455 * d2_a1_b1_mmdt + 0.311882288 * d2_a1_b2_mmdt - 0.131522611 * n2_b1_mmdt - 0.234805732 *
↪ mass_mmdt + 0.648265323 * multiplicity - 0.763080965

      N8 [path=LLR; heap=8; UNREACHABLE (constant ancestor)]
        h_2_8(x) = 0.00394358152 * mass_mmdt - 0.000176803622
        <= 0 -> L2; > 0 -> L3

        L2 [path=LLRL; heap=17; UNREACHABLE (constant ancestor)]
          f_2_2(x) = 0

        L3 [path=LLRR; heap=18; UNREACHABLE (constant ancestor)]
          f_2_3(x) = 0

    N4 [path=LR; heap=4; reachable]
      h_2_4(x) = 0.748689614 * c1_b0_mmdt + 0.669457034 * c2_b1_mmdt + 0.101051343
      <= 0 -> N9; > 0 -> N10

      N9 [path=LRL; heap=9; reachable]
        h_2_9(x) = 0
        <= 0 -> L4; > 0 -> L5; CONSTANT: always LEFT

        L4 [path=LRLL; heap=19; reachable]
          f_2_4(x) = -0.389784207 * zlogz - 0.209597106 * c1_b0_mmdt - 3.08379498 * c1_b1_mmdt - 1.4593788 *
↪ c2_b1_mmdt + 0.0255711958 * d2_b1_mmdt - 0.0574990566 * d2_b2_mmdt + 0.0255711958 * d2_a1_b1_mmdt +
↪ 0.973389446 * d2_a1_b2_mmdt + 0.250244728 * m2_b2_mmdt - 0.782019914 * n2_b1_mmdt + 0.0194565833 *
↪ n2_b2_mmdt + 2.61367386 * mass_mmdt + 0.993769728 * multiplicity + 0.228651881

        L5 [path=LRLR; heap=20; UNREACHABLE (constant ancestor)]
          f_2_5(x) = 0

      N10 [path=LRR; heap=10; reachable]
        h_2_10(x) = -0.615381338 * zlogz - 0.780626703 * multiplicity - 0.248174582
        <= 0 -> L6; > 0 -> L7

        L6 [path=LRRL; heap=21; reachable]
          f_2_6(x) = 0.405855831 * zlogz + 1.19454522 * c1_b0_mmdt - 0.171973875 * c1_b1_mmdt - 3.01430074 *
↪ c2_b1_mmdt + 1.66035781 * c2_b2_mmdt + 0.195722301 * d2_b2_mmdt - 0.470821689 * d2_a1_b2_mmdt +
↪ 0.725279387 * m2_b1_mmdt - 0.0949253938 * m2_b2_mmdt + 1.53492 * n2_b1_mmdt - 0.872843694 * n2_b2_mmdt +
↪ 1.27018591 * mass_mmdt + 0.644659641 * multiplicity + 0.542419575

        L7 [path=LRRR; heap=22; reachable]
          f_2_7(x) = 3.48806978 * zlogz + 7.81025712 * c1_b0_mmdt - 0.95346229 * c1_b2_mmdt + 0.280675132 *
↪ c2_b1_mmdt + 0.0334405957 * d2_b2_mmdt - 1.01384781 * d2_a1_b2_mmdt + 1.85257453 * m2_b1_mmdt -
↪ 0.533779879 * m2_b2_mmdt - 0.647558288 * n2_b1_mmdt + 0.591781799 * n2_b2_mmdt - 1.08156364 * mass_mmdt +
↪ 1.20551885 * multiplicity - 2.42799078

  N2 [path=R; heap=2; reachable]
    h_2_2(x) = 0.168562252 * c2_b2_mmdt - 0.0982298021 * d2_a1_b2_mmdt + 0.37090396 * m2_b1_mmdt +
↪ 0.113712418 * multiplicity - 0.745275257
    <= 0 -> N5; > 0 -> N6

    N5 [path=RL; heap=5; reachable]
      h_2_5(x) = 0.814138721 * mass_mmdt + 0.0531775224 * multiplicity - 0.29709854
      <= 0 -> N11; > 0 -> N12

      N11 [path=RLL; heap=11; reachable]
        h_2_11(x) = 4.94065646e-324
        <= 0 -> L8; > 0 -> L9; CONSTANT: always RIGHT
```

```
        L8 [path=RLLL; heap=23; UNREACHABLE (constant ancestor)]
          f_2_8(x) = 0
        L9 [path=RLLR; heap=24; reachable]
          f_2_9(x) = -0.273060581 * zlogz + 0.214560559 * c1_b0_mmdt + 2.79082823 * c1_b2_mmdt + 0.0872642558
↪  * d2_b1_mmdt + 0.0872642558 * d2_a1_b1_mmdt + 0.0668304306 * d2_a1_b2_mmdt + 0.700471572 * m2_b2_mmdt -
↪ 0.377933505 * n2_b2_mmdt + 0.446788216 * multiplicity + 0.784109979
      N12 [path=RLR; heap=12; reachable]
        h_2_12(x) = 2.06493224e-10 * c1_b2_mmdt + 6.8828261e-11 * c2_b1_mmdt + 2.89598635e-11 * m2_b1_mmdt +
↪ 5.35347149e-11 * m2_b2_mmdt - 1.75510092e-10
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; reachable]
          f_2_10(x) = 0.0172917486 * zlogz + 0.479280898 * c1_b0_mmdt + 0.895727863 * c1_b1_mmdt - 2.47138391
↪  * c1_b2_mmdt + 1.46797745 * c2_b1_mmdt + 0.0674431371 * d2_b1_mmdt - 0.20536907 * d2_b2_mmdt +
↪ 0.0674431371 * d2_a1_b1_mmdt + 0.960087986 * m2_b2_mmdt - 0.399753552 * n2_b1_mmdt - 0.688302486 *
↪ n2_b2_mmdt + 5.79450008 * mass_mmdt + 0.0327603624 * multiplicity - 0.350562803
        L11 [path=RLRR; heap=26; reachable]
          f_2_11(x) = 0.851042201 * c1_b0_mmdt + 1.12087708 * c1_b2_mmdt - 2.14848846 * c2_b1_mmdt +
↪ 0.740207807 * d2_b1_mmdt + 0.740207807 * d2_a1_b1_mmdt + 1.30902686 * m2_b1_mmdt + 4.77523926 * mass_mmdt
↪ + 1.62965024
    N6 [path=RR; heap=6; reachable]
      h_2_6(x) = 0
      <= 0 -> N13; > 0 -> N14; CONSTANT: always LEFT
      N13 [path=RRL; heap=13; reachable]
        h_2_13(x) = 4.94065646e-324
        <= 0 -> L12; > 0 -> L13; CONSTANT: always RIGHT
        L12 [path=RRLL; heap=27; UNREACHABLE (constant ancestor)]
          f_2_12(x) = 0
        L13 [path=RRLR; heap=28; reachable]
          f_2_13(x) = -0.148614563 * zlogz + 1.11851788 * c1_b0_mmdt + 0.96743728 * c1_b1_mmdt +
↪ 0.00568823559 * c2_b2_mmdt + 0.40419135 * d2_b2_mmdt - 1.01497415 * d2_a1_b2_mmdt + 1.11598924 *
↪ m2_b1_mmdt + 1.21950507 * n2_b1_mmdt - 0.144073893 * n2_b2_mmdt + 4.08231683 * mass_mmdt - 0.113704284 *
↪ multiplicity - 0.734827284
      N14 [path=RRR; heap=14; UNREACHABLE (constant ancestor)]
        h_2_14(x) = 0.078412842 * c1_b2_mmdt - 0.128921279
        <= 0 -> L14; > 0 -> L15
        L14 [path=RRRL; heap=29; UNREACHABLE (constant ancestor)]
          f_2_14(x) = 0
        L15 [path=RRRR; heap=30; UNREACHABLE (constant ancestor)]
          f_2_15(x) = 0
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```